

Projet compilation partie 2

21/05/2025

URLI Aristide & PONSON-LISSALDE Romain
MANDRELLI Luca & JULLIEN Baptiste

Sommaire

I. Table des symboles	4
II. Contrôles sémantiques	6
II.1. Statiques	6
II.2. Dynamiques	6
III. Schéma de traduction	7
IV. Gestion de Projet	10
IV.1. Aristide	10
IV.2. Baptiste	10
IV.3. Luca	10
IV.4. Romain	10

I. Table des symboles

La table des symboles est structurée sous forme d'arbre de tableaux. Chaque tableau représente un bloc, et la portée peut être déterminée en examinant les parents dans l'arbre.

Voici un exemple avec le code suivant :

```
def fibonacci(n):  
    if n <= 0:  
        return 0  
    if n == 1:  
        return 1  
    return fibonacci(n-1) + fibonacci(n-2)
```

```
fibonacci(5)
```

La table des symboles correspondante est représentée ci-dessous :

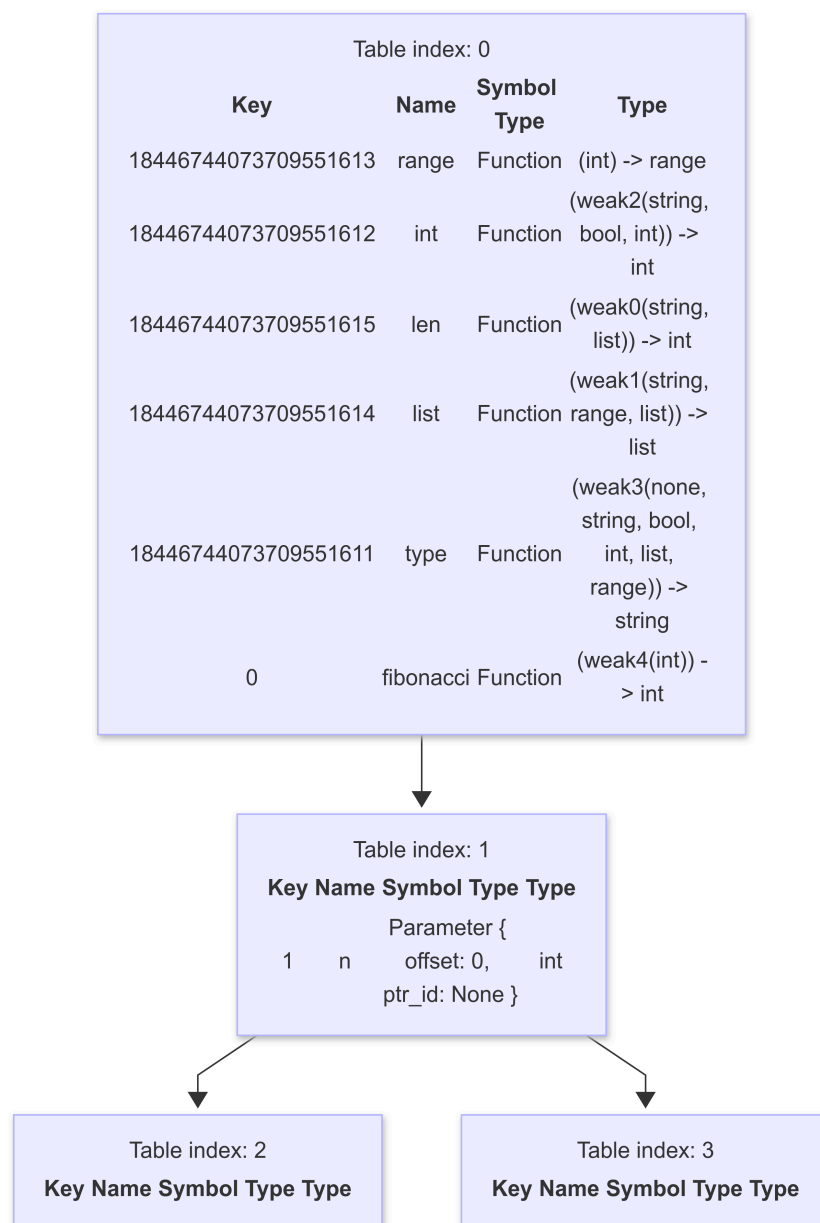


Fig. 1. – Table des symboles pour le code fibonacci

Dans les tableaux, les données sont stockées dans une HashMap, utilisant comme clé un entier identifiant la variable, fourni par la table des identifiants.

Les types de symboles disponibles sont :

- Paramètre
- Fonction
- Variable

Les fonctions sont typées par leur type de retour. Les paramètres et variables peuvent avoir les types suivants :

- None
- Bool
- Int
- String
- List
- Any
- Weak
- Range (Itérateur pour la fonction range)

Chaque type est représenté sur 9 octets : 8 octets pour stocker la donnée et 1 octet pour identifier le type. Cette structure nous permet d'implémenter les types Any et Weak pour le typage dynamique des variables.

Le type Any peut représenter n'importe quel type. Il est notamment utilisé pour le contenu des listes. Un accès à une liste retourne un type Any, les contrôles sémantiques étant effectués dynamiquement à l'exécution.

Le type Weak permet de réaliser des intersections ou des unions de types pour l'inférence de types à la compilation. Par exemple :

- Une opération « - » entre deux variables n'est possible qu'entre deux Int, le type résultant sera donc un Int
- Si une fonction retourne soit un booléen soit une chaîne de caractères Weak(String, Bool), une opération « - » avec ce retour générera une erreur statique

Le type Range est un type interne contenant un Int. Il reproduit le comportement d'un itérateur Python dans une boucle for et la fonction list.

Lors de la compilation, les pointeurs des variables et paramètres sont intégrés dans la table des symboles. Ces pointeurs représentent la position dans la pile.

II. Contrôles sémantiques

II.1. Statiques

À la compilation, nous pouvons détecter plusieurs types d'erreurs sémantiques. En particulier, nous inférons les types de chaque identifiant et vérifions la compatibilité des types lors des opérations binaires.

Exemple : Addition impossible entre un Int et une List

```
0 + [1, 2, 3] # Erreur
```

Exemple : Addition impossible entre un Int et un Weak(String, List)

```
def f():  
    if 0:  
        return [1, 2, 3]  
    else:  
        return "toto"  
  
# f est typé Weak(String, List)  
0 + f() # Erreur car Int + Weak(String, List) incompatible
```

II.2. Dynamiques

Certains aspects du programme ne peuvent être vérifiés qu'à l'exécution. Nous avons donc implémenté plusieurs contrôles dynamiques. Par exemple, la vérification que les indices d'accès à une liste sont compris dans sa taille.

Exemple d'erreur dynamique :

```
a = ["test"]  
# a[0] est typé Any  
a[0] + 1 # Erreur Dynamique String + Int
```

III. Schéma de traduction

Voici un exemple de code en mini-python, implémentant la fonction fibonacci réursive :

```
def fibonacci(n): # ■
    if n <= 0: # 0
        return 0 # ▲
    if n == 1: # 1
        return 1 # ▼
    return fibonacci(n-1) + fibonacci(n-2)

fibonacci(5) # ■
```

```
define i32 @main() {
entry:
    %function_call_fibonacci = call %dynamic_type_struct
@__smolpp_user_f_fibonacci(%dynamic_type_struct { i8 4, i64 5 }) ; ■
    ret i32 0
}

define %dynamic_type_struct @__smolpp_user_f_fibonacci(%dynamic_type_struct %0) { ;
■
function_entry:
    %alloca_param_n = alloca %dynamic_type_struct, align 8
    store %dynamic_type_struct %0, ptr %alloca_param_n, align 4
    %load_n = load %dynamic_type_struct, ptr %alloca_param_n, align 4
    %value_field = extractvalue %dynamic_type_struct %load_n, 1
    %lte = icmp sle i64 %value_field, 0
    %int_cast = sext il %lte to i64
    %with_value = insertvalue %dynamic_type_struct { i8 2, i64 undef }, i64
%int_cast, 1
    %value_field1 = extractvalue %dynamic_type_struct %with_value, 1
    %bool_if = trunc i64 %value_field1 to i1
    br il %bool_if, label %then, label %else ; 0

then:
    ; preds = %function_entry
    ret %dynamic_type_struct { i8 4, i64 0 } ; ▲

else:
    ; preds = %function_entry
    br label %merge

merge:
    ; preds = %else
    %load_n2 = load %dynamic_type_struct, ptr %alloca_param_n, align 4
    %value_field3 = extractvalue %dynamic_type_struct %load_n2, 1
    %eq = icmp eq i64 %value_field3, 1
    %int_cast4 = sext il %eq to i64
    %with_value5 = insertvalue %dynamic_type_struct { i8 2, i64 undef }, i64
```

```

%int_cast4, 1
  %value_field9 = extractvalue %dynamic_type_struct %with_value5, 1
  %bool_if10 = trunc i64 %value_field9 to i1
  br i1 %bool_if10, label %then6, label %else7 ; 1

then6:                                     ; preds = %merge
  ret %dynamic_type_struct { i8 4, i64 1 } ; ▼

else7:                                     ; preds = %merge
  br label %merge8

merge8:                                    ; preds = %else7
  %load_n11 = load %dynamic_type_struct, ptr %alloca_param_n, align 4
  %value_field12 = extractvalue %dynamic_type_struct %load_n11, 1
  %sub = sub i64 %value_field12, 1
  %with_value13 = insertvalue %dynamic_type_struct { i8 4, i64 undef }, i64 %sub, 1
  %function_call_fibonacci = call %dynamic_type_struct
@_smolpp_user_f_fibonacci(%dynamic_type_struct %with_value13)
  %load_n14 = load %dynamic_type_struct, ptr %alloca_param_n, align 4
  %value_field15 = extractvalue %dynamic_type_struct %load_n14, 1
  %sub16 = sub i64 %value_field15, 2
  %with_value17 = insertvalue %dynamic_type_struct { i8 4, i64 undef }, i64 %sub16,
1
  %function_call_fibonacci18 = call %dynamic_type_struct
@_smolpp_user_f_fibonacci(%dynamic_type_struct %with_value17)
  %value_field19 = extractvalue %dynamic_type_struct %function_call_fibonacci, 1
  %value_field20 = extractvalue %dynamic_type_struct %function_call_fibonacci18, 1
  %add = add i64 %value_field19, %value_field20 ; +
  %with_value21 = insertvalue %dynamic_type_struct { i8 4, i64 undef }, i64 %add, 1
  ret %dynamic_type_struct %with_value21
}

```

Voici un autre exemple de code en mini-python illustrant les opérateurs logiques :

```

println(1 or [1,3])
println(1 and [1,3])
println(1 or True and [1] or "A")

```

Sortie :

```

1
[1, 3]
1

```

Les opérateurs `and` et `or` sont évalués paresseusement : si le premier opérande est suffisant pour déterminer le résultat, le second n'est pas évalué. Ils retournent la dernière valeur qui a permis de déterminer le résultat.

Voici un extrait de l'assembleur LLVM correspondant :


```

define i32 @main() {
entry:
    %and_or_bool_cast_call = call i1 @__smolpp_f_bool_cast(%dynamic_type_struct { i8
4, i64 1 })
    br i1 %and_or_bool_cast_call, label %finish_block, label %compute_right_block

compute_right_block:                                ; preds = %entry
    %0 = trunc i64 2 to i32
    %mallocsize = mul i32 %0, ptrtoint (ptr getelementptr (%dynamic_type_struct, ptr
null, i32 1) to i32)
    %list_heap_array = tail call ptr @malloc(i32 %mallocsize)
    %list = tail call ptr @malloc(i32 ptrtoint (ptr getelementptr (%list_struct, ptr
null, i32 1) to i32))
    %struct_len_ptr = getelementptr inbounds %list_struct, ptr %list, i32 0, i32 0
    %struct_capa_ptr = getelementptr inbounds %list_struct, ptr %list, i32 0, i32 1
    %struct_array_ptr = getelementptr inbounds %list_struct, ptr %list, i32 0, i32 2
    store i64 0, ptr %struct_len_ptr, align 4
    store i64 2, ptr %struct_capa_ptr, align 4
    store ptr %list_heap_array, ptr %struct_array_ptr, align 8
    %list_ptr = ptrtoint ptr %list to i64
    %with_value = insertvalue %dynamic_type_struct { i8 16, i64 undef }, i64
%list_ptr, 1
    %len_ptr = getelementptr inbounds %list_struct, ptr %list, i32 0, i32 0
    store i64 2, ptr %len_ptr, align 4
    %array_ptr_ptr = getelementptr inbounds %list_struct, ptr %list, i32 0, i32 2
    %array_ptr = load ptr, ptr %array_ptr_ptr, align 8
    %elt_ptr = getelementptr %dynamic_type_struct, ptr %array_ptr, i32 0
    store %dynamic_type_struct { i8 4, i64 1 }, ptr %elt_ptr, align 4
    %elt_ptr1 = getelementptr %dynamic_type_struct, ptr %array_ptr, i32 1
    store %dynamic_type_struct { i8 4, i64 3 }, ptr %elt_ptr1, align 4
    br label %finish_block

finish_block:                                       ; preds = %compute_right_block,
%entry
    %llvm_compute_and_or_phi = phi %dynamic_type_struct [ { i8 4, i64 1 }, %entry ],
[ %with_value, %compute_right_block ]
    call void @__smolpp_f_generic_print(%dynamic_type_struct
%llvm_compute_and_or_phi)
    %line_return = call i32 (ptr, ...) @printf(ptr @__smolpp_g_line_return)
    %and_or_bool_cast_call2 = call i1 @__smolpp_f_bool_cast(%dynamic_type_struct { i8
4, i64 1 })
    br i1 %and_or_bool_cast_call2, label %compute_right_block3, label %finish_block4

```

L'instruction phi sélectionne la valeur en fonction du bloc prédécesseur :

- Si le bloc prédécesseur est entry, la valeur est { i8 4, i64 1 }
- Sinon, la valeur est celle de with_value, soit { i8 16, i64 undef }

Les opérateurs and et or peuvent retourner n'importe quel type. Ils sont donc typés Any et nécessitent des contrôles sémantiques dynamiques.

IV. Gestion de Projet

IV.1. Aristide

Typage : typage statique (10h) ; typage dynamique (3h)

Contrôles sémantiques statiques : vérification des types statiques (5h) ;

Contrôles sémantiques statiques : assertions des types pour les opérations (3h) ;

Assembleur : print de base (1h) ; print générique (1h) ; Variables (4h) ; Strings (3h) ; Listes (5h) ; arithmétique générique (2h) ; type Weak (17h) ; boucle while (1h) ; fonctions de la bibliothèque standard (input) (2h) ; concaténation des strings et des listes (5h) ; comparaison des strings (1h)

IV.2. Baptiste

Contrôles sémantiques dynamiques : comparaison générique (10h) ; « and », « or », « not », « if » -> cast de tout type en booléen (2h) ; vérification boucle for : list ou range (4h)

Configuration LLVM : configuration des bibliothèques et recherches, choix entre inkwell et llvml-ir (20h) ; configuration du linker LLVM pour Linux (20h) ; adaptation du CLI pour compilation et exécution (5h)

Assembleur : structure de base : fonction main, setup initial (4h) ; expressions (2h) ; boucles : boucle « for » (12h) ; fonctions « and », « or », « not » (4h) ; fonctions « len », « list », « range » (6h) ; création d'une bibliothèque standard (3h)

IV.3. Luca

Table des symboles : remplissage de la TDS depuis l'AST (7h) ; gestion des offsets (5h)

Contrôles sémantiques statiques : portée des variables (4h)

Configuration LLVM : adaptation de la configuration du linker LLVM pour macOS en préservant la compatibilité Linux (4h)

Assembleur : opérations binaires (3h) ; comparaisons statiques (4h) ; fonctions internes : range et list (5h) ; affichage des erreurs d'exécution (2h) ; localisation des erreurs d'exécution (5h) ; fonction interne type (5h)

IV.4. Romain

Table des symboles : affinage des structures de données (2h)

Contrôles sémantiques statiques : vérification des arguments et retours de fonctions (3h)

Assembleur : typage dynamique des fonctions (4h) ; appel de fonctions (2h) ; branchement if-else (2h) ; accès et assignation de liste (3h) ; comparaison de listes (6h) ; appels de fonctions mutuellement récursives (1h) ; fonction interne int (3h) ; surcharge de l'opérateur plus (4h)